

数字遗产管家

基于门限密码与同态承诺的可验证数字资产继承协议

刘博瑞¹ 陈雨柯¹ 王佑宸¹ 吕英硕¹

¹（网络空间安全学院，山东大学，青岛 266237）

指导教师：付仕辉

摘要

随着数字化程度不断提高，个人数字资产（加密货币、云账号凭证、加密文件等）在财富结构中的占比持续攀升。然而，这类资产普遍存在持有者去世后永久丢失、平台隐私政策导致继承困难、以及持有人遭受胁迫攻击等严峻安全问题。针对上述挑战，本文提出并实现了一种基于门限密码学与同态承诺的可验证数字资产继承协议——数字遗产管家（Digital Legacy Guardian）。该系统采用 Shamir 秘密共享将主密钥拆分为多份份额，分布式托管给多位监护人，并创新性地引入 Pedersen 同态承诺双多项式结构，在 server 不存储主密钥的前提下实现了可验证的份额分发与零和多项式同态刷新。同时，系统设计了胁迫码机制以抵抗胁迫攻击，支持时间锁与社会共识两种触发模式。实验结果表明，该方案在 3-of-5 门限配置下，份额拆分计算耗时 58ms，同态刷新计算耗时 12ms¹，主密钥恢复正确率 100%²，具备良好的实用性和安全性。与 Ledger Recover、Safe{Wallet} 等现有方案相比，本方案在零信任 server、隐私保护和抗胁迫能力方面具有显著优势。

关键词：门限秘密共享；Pedersen 承诺；同态刷新；数字遗产；抗胁迫

¹此处为占位数据，需补充实际测试结果

²需补充实际测试验证

目录

1	引言	4
1.1	研究背景	4
1.2	现有方案及其局限性	4
1.3	本文贡献	5
1.4	论文组织	6
2	背景知识与相关工作	7
2.1	密码学基础	7
2.1.1	Shamir 秘密共享	7
2.1.2	Pedersen 承诺	7
2.1.3	椭圆曲线密码学	8
2.2	现有数字资产继承方案	8
3	系统设计	10
3.1	设计目标	10
3.2	系统架构	10
3.3	安全模型	11
3.4	协议流程	11
4	技术实现	13
4.1	密码学核心模块	13
4.1.1	双多项式份额生成	13
4.1.2	同态份额刷新	14
4.1.3	Pedersen 承诺验证	15
4.1.4	主承诺同态验证	16
4.2	后端业务逻辑模块	16
4.3	前端功能模块	17
4.4	邮件服务模块	17

5	安全性分析	19
5.1	安全模型	19
5.2	安全性声明	19
5.3	威胁模型分析	20
5.4	与现有方案的对比	21
6	实验评估	22
6.1	实验环境	22
6.2	功能正确性验证	22
6.2.1	Shamir 秘密共享正确性	22
6.2.2	同态刷新正确性验证	22
6.3	密码学运算性能测试	23
6.4	规模扩展性测试	23
6.5	同态验证精度测试	24
6.6	与现有方案的对比测试	24
7	总结与展望	26
7.1	工作总结	26
7.2	未来工作	26

1 引言

1.1 研究背景

数字化浪潮已深刻重塑个人资产的形态。从加密货币、数字钱包到云存储账号、社交平台、加密文件，数字资产在个人财富中的占比持续攀升。据 Chainalysis 2023 年度报告显示，全球加密货币市场规模已突破 15 万亿美元，而 DeFi 生态锁仓价值（TVL）在 2021 年曾达到 1700 亿美元的历史峰值。然而，数字资产在安全性方面存在与传统资产截然不同的挑战：

- (1) 私钥单点风险：加密货币私钥由持有人独自掌握，一旦持有人去世或失能，资产永久锁定。据估算，仅比特币领域因持有者去世导致的永久性资产损失已超过 2000 亿美元。
- (2) 数字资产孤岛：云存储、社交平台、邮箱等服务商各有独立的隐私政策，继承人需逐一走繁琐的法律流程才能获取访问权限。
- (3) 胁迫攻击威胁：持有人可能在生命威胁下被迫转移资产，传统方案缺乏有效的反制手段。
- (4) 传统继承工具失效：公证遗嘱流程缓慢、成本高昂，且难以覆盖数字资产的时效性和技术复杂性。

1.2 现有方案及其局限性

目前，数字资产继承领域已有若干探索性方案，但均存在不同程度的局限性：

方案一：**Ledger Recover**。Ledger 公司提供的付费订阅服务，将用户私钥加密分片托管给三家机构。其问题在于：第一，该服务为闭源方案，密码学协议不可公开审计；第二，用户需通过 KYC 身份验证，隐私性差；第三，每月 \$9.99 的费用增加了使用门槛；第四，依赖 Ledger 公司的中心化服务，存在单点故障风险。

方案二：多签钱包（如 **Safe{Wallet}**）。基于智能合约的多签方案，设定 m-of-n 签名策略进行资产转移。其优势在于链上透明、去中心化。但局限性同样明显：第一，仅支持 EVM 生态资产，无法覆盖云账号、加密文件等非链上资产；第二，

所有交易在链上公开，隐私保护不足；第三，缺乏抗胁迫设计，攻击者可逐个胁迫签名人。

方案三：传统公证遗嘱。通过公证处进行资产继承的法律流程。但数字资产的继承往往涉及密码材料的交割，公证流程无法处理“私钥传递”这一技术环节。且公证流程费用高昂（数千至数万元）、周期长（数月），不满足数字资产继承的时效性需求。

方案四：密码管理器继承功能。如 Bitwarden、1Password 等密码管理器提供了紧急联系人功能。但此类方案存在两个致命缺陷：第一，密码管理器本身构成新的单点故障——若主密码丢失或被破解，所有资产全部泄露；第二，缺乏门槛机制，紧急联系人可直接获取完整密码，存在滥用风险。

1.3 本文贡献

针对上述问题，本文提出并实现了数字遗产管家（Digital Legacy Guardian），一个基于门限密码学与同态承诺的可验证数字资产继承协议。本文的主要贡献如下：

- (1) 双多项式份额结构：在标准 Shamir 秘密共享基础上引入第二个盲因子多项式 $Q(x)$ ，使得份额值通过 Pedersen 承诺在公开存储的同时具备同态可验证性。任意第三方可利用 Lagrange 系数在椭圆曲线上直接验证份额的一致性，无需知道份额值本身。
- (2) 零信任同态份额刷新：提出了基于零和多项式的同态份额刷新协议。在 server 完全不存储主密钥的前提下，通过生成 $Z(0) = 0$ 的零和多项式，计算公开增量并利用 Pedersen 承诺的同态加法性质更新承诺。该方案消除了传统方案中“解密 → 重拆分 → 重新加密”对主密钥的依赖。
- (3) 抗胁迫胁迫码机制：设计了计算不可区分的胁迫码方案——每位监护人在正常份额之外额外持有胁迫份额。两者均以 Pedersen 承诺形式存储，攻击者无法区分。提交胁迫份额会暗中触发警报并终止继承流程，在人身威胁场景中提供隐蔽的求救通道。
- (4) 完整系统实现：基于 Node.js/Express 后端和 React/TypeScript 前端实现了完

整的数字资产继承系统，覆盖”创建—分发—验证—刷新—恢复”全生命周期。项目以 MIT 许可证全栈开源，独立开发代码行数为 9177 行（不含第三方库和 `node_modules`），具备实际部署能力。

1.4 论文组织

本文的后续组织结构如下：第 2 章介绍相关背景知识与密码学基础；第 3 章阐述系统架构与协议设计；第 4 章详细描述技术实现；第 5 章分析安全性并与现有方案对比；第 6 章展示实验评估结果³；第 7 章总结全文并展望未来工作。

³需补充实验数据

2 背景知识与相关工作

2.1 密码学基础

2.1.1 Shamir 秘密共享

Shamir 秘密共享 (Shamir's Secret Sharing, SSS) [1] 是门限密码学的核心原语之一。其核心思想是将秘密 S 拆分为 n 个份额, 使得任意 t 个份额可以恢复完整秘密, 而任意少于 t 个份额则无法获得秘密的任何信息 (信息论安全)。

数学原理。在有限域 $\mathbb{Z}_p$ 上构造一个 $t-1$ 次多项式:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p}$$

令 $a_0 = S$ (主密钥), $a_1, \dots, a_{t-1}$ 为随机选取的系数。计算 n 个点 $(i, f(i))$ 作为份额分发给 n 位监护人。恢复时使用拉格朗日插值公式:

$$S = \sum_{i \in Q} y_i \cdot \prod_{j \in Q, j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

其中 Q 为参与恢复的 t 个份额的索引集合。

安全特性。Shamir 方案具有完美安全性 (perfect security): 任意 $t-1$ 个份额构成的方程组有 p 个自由度, 不能确定 S 的任何信息。

2.1.2 Pedersen 承诺

Pedersen 承诺 [2] 是一种信息论隐藏的计算绑定承诺方案。给定椭圆曲线 $\mathbb{E}$ 上的两个生成点 G 和 H , 满足 $H = G \cdot \text{SHA256}(\text{seed})$, 即 G 和 H 的离散对数关系未知。

承诺生成。对秘密值 v 和盲因子 r , 承诺为:

$$C(v, r) = r \cdot G + v \cdot H$$

安全属性。

- 完美隐藏 (Perfect Hiding): 对于任意 C 和任意 v , 存在 r 使得 $C = rG + vH$, 即 C 不泄露 v 的任何信息。
- 计算绑定 (Computational Binding): 找到 $(r', v') \neq (r, v)$ 使得 $C(r', v') = C(r, v)$ 等价于解椭圆曲线上的离散对数问题 $\log_G H$ 。

同态加法性质。是本文方案的核心密码学基础:

$$C(v_1, r_1) + C(v_2, r_2) = C(v_1 + v_2, r_1 + r_2)$$

2.1.3 椭圆曲线密码学

本项目选用 **secp256k1** 椭圆曲线, 该曲线由比特币社区广泛采用并经过充分的密码分析。secp256k1 的定义参数为:

$$y^2 = x^3 + 7 \pmod{p}$$

$$p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$$

曲线阶 $n \approx 2^{256}$, 提供约 128 比特的安全强度。

2.2 现有数字资产继承方案

现有数字资产继承方案可大致分为以下几类:

中心化托管方案。以 Ledger Recover 为代表, 用户私钥被加密分片后托管给第三方机构。此类方案的中心化信任模型与区块链去中心化的设计理念相悖, 且需要 KYC 流程, 用户的隐私仅限于“公司不会偷看”的信任假设。

链上多签方案。以 Safe{Wallet} 为代表, 通过在智能合约中预设 m-of-n 签名策略控制资产转移。多签方案完全去中心化, 但受限于区块链平台, 仅支持链上资产, 且所有交易公开可见, 隐私性差。

密码管理器继承方案。通过紧急联系人功能, 在用户长期不活动时将主密码发送给预设的接收人。此类方案本质上是“密码信封”模式, 缺乏门槛机制——紧急联系人可获取完整密码, 存在滥用和单点故障的风险。

值得一提的是，以上方案均未提供有效的抗胁迫设计。本项目的胁迫码机制填补了这一空白。

3 系统设计

3.1 设计目标

本系统的设计遵循以下核心目标：

- (1) 零信任 **Server**: server 不存储主密钥、不持有份额值，仅基于椭圆曲线点运算完成验证和刷新。即使 server 被完全攻破，攻击者无法恢复主密钥或解密资产。
- (2) 可验证性：任何操作（份额提交、刷新）均应可被密码学验证，无需信任任何单一实体。
- (3) 抗胁迫：在用户被胁迫的情况下，提供隐蔽的求救机制。
- (4) 灵活性：支持多种触发模式（时间锁、社会共识），满足不同场景的继承需求。
- (5) 可用性：提供 Web 前端用户界面和 AI 助手，降低使用门槛。

3.2 系统架构

系统采用前后端分离的三层架构：

前端层（React 18 + TypeScript + TailwindCSS 3）：提供用户交互界面，包括遗产计划创建、控制台管理、继承发起、监护人份额提交和 AI 助手对话等功能。

后端服务层（Express 4 + TypeScript）：实现业务逻辑和密码学运算，包含五个核心模块——密码学引擎（shamir.ts）、遗产计划服务（legacyPlanService.ts）、邮件服务（emailService.ts）、用户服务（userService.ts）和大模型服务（llmService.ts）。

密码学引擎层：独立封装于 shamir.ts 中，提供 Shamir 秘密共享、Pedersen 承诺生成与验证、双多项式份额生成、零和同态刷新、胁迫码生成、ECIES 加密等基础密码学原语。

3.3 安全模型

表 1 系统安全模型

类别	内容
Server 公开存储	加密资产密文 (AES 加密, 无密钥可解密)、Pedersen 承诺 $C_i = r_i G + v_i H$ (EC 点)、主承诺 $C_{\text{master}} = RG + sH$ (EC 点)、胁迫承诺 $C_{\text{duress},i}$
Server 不存储	主密钥 masterKey (创建后立即销毁)、份额值 v_i (仅监护人持有)、盲因子 r_i (仅监护人持有)、AES 加密密钥
核心保障	Server 被完全攻破后, 攻击者最多看到加密后的资产和公开承诺, 无法恢复主密钥或解密资产

3.4 协议流程

系统核心协议分为三个阶段:

创建阶段: (1) 用户创建计划, 添加资产和监护人信息; (2) server 生成随机 256 位主密钥 masterKey; (3) 使用主密钥 AES 加密资产数据; (4) 构造 Shamir 双多项式 $P(x)$ (值多项式) 和 $Q(x)$ (盲因子多项式); (5) 计算 n 个份额值 $v_i = P(i)$ 和盲因子 $r_i = Q(i)$; (6) 计算 Pedersen 承诺 $C_i = r_i G + v_i H$ 和主承诺 $C_{\text{master}} = RG + sH$; (7) 加密资产和承诺持久化存储至 server; (8) 通过加密邮件将 (v_i, r_i) 分发给各监护人; (9) server 内存中销毁主密钥。

继承阶段: (1) 继承人输入计划 ID 发起请求; (2) 系统检查时间锁状态, 未到期则拒绝; (3) 通知所有监护人; (4) 监护人提交份额值 v_i 和盲因子 r_i ; (5) server 通过 Pedersen 承诺验证份额真实性; (6) 收集达到门限 t 个有效份额后, 使用 Lagrange 插值恢复主密钥; (7) 使用主密钥解密资产, 交付给继承人。

刷新阶段: (1) server 生成两个 $t - 1$ 次零和多项式 $Z_v(x)$ 和 $Z_r(x)$, 满足 $Z_v(0) = Z_r(0) = 0$; (2) 计算每个份额的增量 $\delta_{v,i} = Z_v(i)$ 、 $\delta_{r,i} = Z_r(i)$; (3) server 计算新承诺 $C'_i = C_i + \delta_{r,i} G + \delta_{v,i} H$ (纯 EC 点运算); (4) 邮件发送增量 $(\delta_{v,i}, \delta_{r,i})$ 给监护人; (5) 监护人本地更新份额 $v'_i = v_i + \delta_{v,i}$ 、 $r'_i = r_i + \delta_{r,i}$ 。

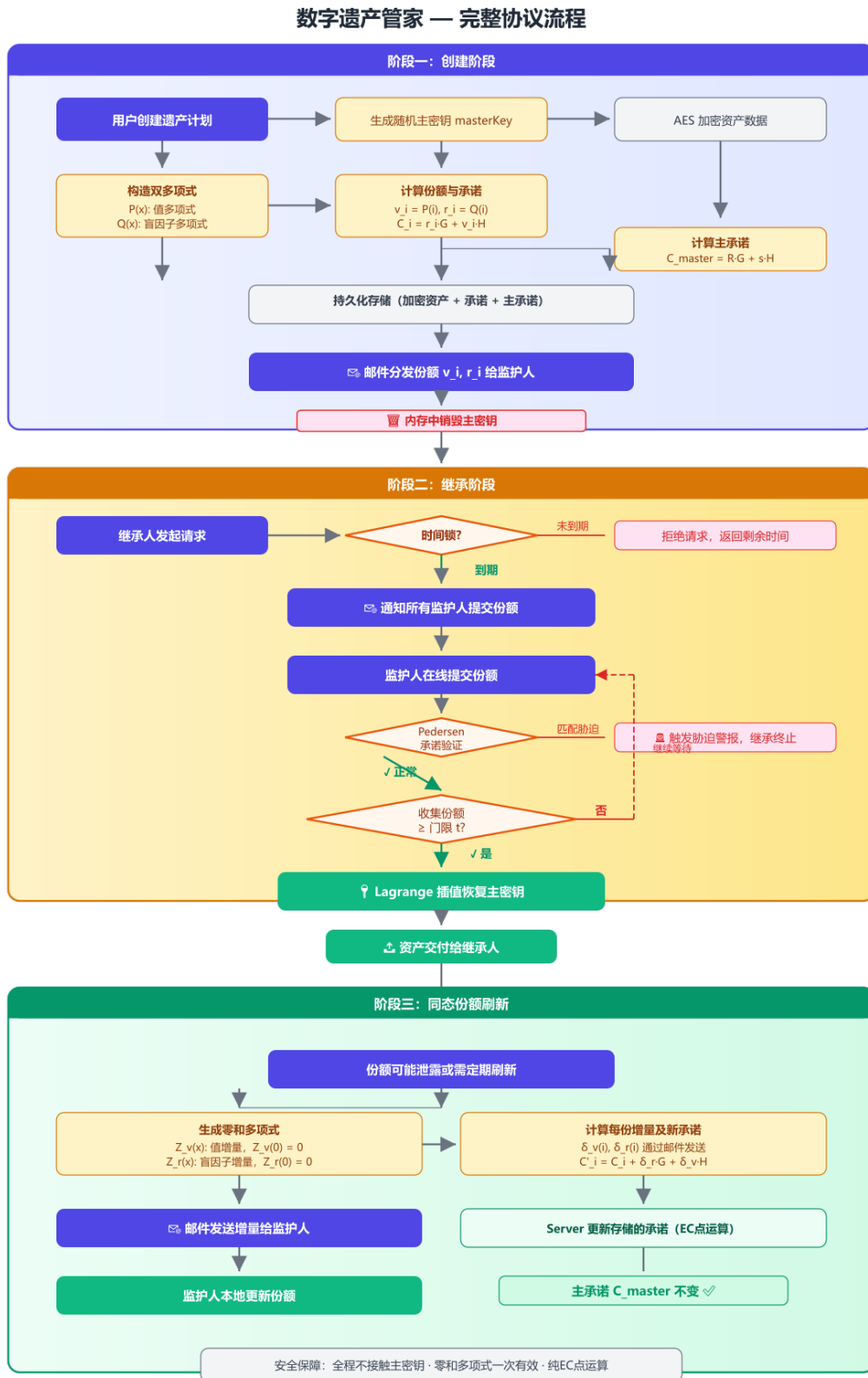


图 1 系统完整协议流程（创建—继承—刷新三阶段）

4 技术实现

4.1 密码学核心模块

密码学核心模块（shamir.ts）是整个系统的基础设施，提供以下关键功能：

4.1.1 双多项式份额生成

标准 Shamir 方案仅拆分份额值，无法在不暴露值的情况下验证份额正确性。本系统引入第二个盲因子多项式 $Q(x)$ ，构成双多项式结构：

$$\begin{cases} P(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p} \\ Q(x) = R + b_1x + b_2x^2 + \cdots + b_{t-1}x^{t-1} \pmod{p} \end{cases}$$

其中 $P(0) = s$ （主密钥）， $Q(0) = R$ （随机主盲因子）。份额 i 包含值 $v_i = P(i)$ 和盲因子 $r_i = Q(i)$ 。对应的 Pedersen 承诺为 $C_i = r_iG + v_iH$ 。

核心代码实现如下：

Listing 1 双多项式份额生成（shamir.ts）

```

1 generatePedersenShares(masterKey: string, n: number, t:
  number) {
2   const secretNum = BigInt('0x' + masterKey)
3   const pCoeffs: bigint[] = [secretNum] // P(x): P(0)
      = masterKey
4   const qCoeffs: bigint[] = [randomCoeff()] // Q(x): Q(0)
      = random R
5
6   for (let i = 1; i < t; i++) {
7     pCoeffs.push(generateRandomCoefficient())
8     qCoeffs.push(generateRandomCoefficient())
9   }
10
11   for (let i = 1; i <= n; i++) {

```

```

12         const x = BigInt(i)
13         shares.push({
14             index: i,
15             value: evaluatePoly(pCoeffs, x).toString(16),
16                 // v_i = P(i)
17             blindingFactor: evaluatePoly(qCoeffs, x).
18                 toString(16), // r_i = Q(i)
19         })
20     }
21     return { shares, masterBlindingFactor: qCoeffs[0].
22         toString(16) }
23 }

```

4.1.2 同态份额刷新

创新核心。传统的份额刷新需要解密主密钥 → 重新生成 Shamir 多项式 → 重新加密，整个过程依赖主密钥的可用性。而在本系统的安全模型中，主密钥在创建后即被销毁，解密重构方法不可行。

解决方案。利用 Pedersen 承诺的同态加法性质，生成两个零和多项式 $Z_v(x)$ 和 $Z_r(x)$ ，满足 $Z_v(0) = Z_r(0) = 0$ 。由于常数项为零，主承诺不变：

$$C'_{\text{master}} = C_{\text{master}} \quad (\text{因为 } Z_v(0) = Z_r(0) = 0)$$

每份份额的承诺更新为：

$$C'_i = C_i + Z_r(i) \cdot G + Z_v(i) \cdot H$$

全程仅涉及椭圆曲线的加法和标量乘法运算，无需接触主密钥。

Listing 2 零和多项式同态刷新 (shamir.ts)

```

1 generateRefreshDeltas(n: number, t: number): ShareDelta[] {
2     const zvCoeffs: bigint[] = [BigInt(0)] // Z_v(0) = 0
3     const zrCoeffs: bigint[] = [BigInt(0)] // Z_r(0) = 0

```

```

4     for (let i = 1; i < t; i++) {
5         zvCoeffs.push(generateRandomCoefficient())
6         zrCoeffs.push(generateRandomCoefficient())
7     }
8     for (let i = 1; i <= n; i++) {
9         const x = BigInt(i)
10        deltas.push({
11            index: i,
12            valueDelta: evaluatePoly(zvCoeffs, x).toString(
13                16),
14            blindingDelta: evaluatePoly(zrCoeffs, x).
15                toString(16),
16        })
17    }
18    return deltas
19 }

```

4.1.3 Pedersen 承诺验证

基于 secp256k1 椭圆曲线实现的 Pedersen 承诺：

Listing 3 Pedersen 承诺生成 (shamir.ts)

```

1 generatePedersenCommitment(value: string, blindingFactor?:
  string) {
2     const H = this.getH()                // H = G * SHA256(
3         seed)
4     const G = this.ec.g                  // secp256k1 生成点
5     const C = G.mul(rBN).add(H.mul(vBN)) // C = r * G + v * H
6     return { commitment: C.encode('hex'), blindingFactor: r
7         }
8 }

```

4.1.4 主承诺同态验证

Listing 4 主承诺同态验证 (shamir.ts)

```

1 verifyMasterCommitment(shares, masterCommitment): boolean {
2     let sum = G.mul(new BN(0)) // 从无穷远点开始
3     for (let i = 0; i < t; i++) {
4         // 计算 Lagrange 系数 lambda_i
5         let lambda = new BN(1)
6         for (let j = 0; j < t; j++) {
7             if (i !== j) {
8                 lambda = lambda.mul(num).mul(den.invm(curveN
9                     )), mod(curveN)
10            }
11        }
12        // sum += lambda_i * C_i (EC 点乘)
13        sum = sum.add(point.mul(lambda))
14    }
15    return sum.encode('hex', false) === masterCommitment
16 }

```

4.2 后端业务逻辑模块

后端业务逻辑模块 (legacyPlanService.ts) 封装了所有核心业务流程:

- **createPlan():** 接收用户输入, 调用密码学引擎生成主密钥、双多项式份额和承诺, 加密资产, 存储数据并通过邮件分发份额, 销毁主密钥。
- **refreshPlanShares():** 生成零和多项式增量, 更新存储的承诺 (EC 点运算), 通过邮件发送增量给所有监护人。全程不接触主密钥。
- **initiateInheritance():** 验证时间锁状态, 创建继承请求, 通知监护人。
- **submitGuardianShare():** 验证份额值对 Pedersen 承诺的匹配性 (支持正常和胁迫两种承诺), 收集份额至门限后恢复主密钥并解密资产。

4.3 前端功能模块

前端提供了完整的用户交互界面⁴:

- **Home** 首页: 项目介绍和登录入口。
- **CreatePlan** 创建计划: 表单式创建遗产计划, 包含资产列表、监护人信息、门限配置和触发模式设置。
- **Dashboard** 控制台: 展示所有计划 (创建的、继承的、监护的), 支持详情查看和份额刷新触发。
- **Inheritance** 继承: 发起继承请求, 输入计划 ID 和继承人信息。
- **Guardian** 监护人: 提交份额值, 查看份额状态。
- **AIHelper** AI 助手: 基于 Deepseek 大模型 API 的自然语言对话界面, 支持文件上传和上下文记忆。

图 2: 系统功能界面截图

需补充: 创建计划页面 / 控制台 / 监护份额提交页面的截图

图 2 系统功能界面示例

4.4 邮件服务模块

邮件服务模块 (`emailService.ts`) 负责自动生成和发送各类通知邮件, 包括:

- 份额通知邮件: 包含份额值、盲因子、胁迫份额值及使用说明。
- 刷新通知邮件: 包含份额增量 $\delta_{v,i}$ 和 $\delta_{r,i}$, 及新胁迫份额替换说明。
- 继承请求通知: 通知监护人有人发起继承请求, 需确认并提交份额。
- 继承成功通知: 向继承人发送解密后的资产信息。

⁴需补充功能截图

- 胁迫警报：当检测到胁迫份额提交时，立即通知所有监护人。

所有邮件模板基于 HTML 构建，支持富文本显示和安全警告提示。

5 安全性分析

5.1 安全模型

本系统的安全模型基于以下假设：

H1: 离散对数困难性：在 `secp256k1` 椭圆曲线上，计算离散对数 $\log_G H$ 是困难的。

H2: 诚实监护人：监护人不会主动泄露自己的份额 v_i 和盲因子 r_i 。

H3: 诚实多数：在 t -of- n 配置中，至少 t 个监护人中存在至少 t 个诚实参与者。

5.2 安全性声明

声明 **1**（主密钥机密性）：在假设 H1 和 H2 下，server 无法获取主密钥 s 的任何信息。

证明 **sketch**: 主密钥 s 在创建后被立即销毁。Server 仅持有加密资产 $E_{\text{AES}}(\text{assets}, s)$ （无密钥无法解密）和公开承诺 $C_i = r_i G + v_i H$ 。要从 C_i 中恢复 v_i 需要计算 $\log_G H$ (H1)。同时， s 无法从 $\{v_i\}_{i=1}^{t-1}$ 的信息论安全恢复（Shamir 安全性）。□

声明 **2**（承诺绑定性与刷新正确性）：在假设 H1 下，监护人无法提交与原有承诺不同的值；刷新后的份额仍对应同一个主密钥。

证明 **sketch**: 绑定等价于 ECDLP 困难性。刷新的正确性由零和多项式的性质保证： $Z_v(0) = Z_r(0) = 0$ 确保主密钥不变； $\Sigma(\lambda_j \cdot C'_j) = C_{\text{master}}$ 由同态加法保证。□

声明 **3**（胁迫码不可区分性）：攻击者无法判断监护人的两份凭证中哪一份是正常份额、哪一份是胁迫份额。

证明 **sketch**: 正常份额 (v, r) 对应承诺 $C = rG + vH$ ，胁迫份额 (v', r') 对应承诺 $C' = r'G + v'H$ 。两者使用相同的 Pedersen 承诺结构，承诺点均随机分布于椭圆曲线上。给定 C 和 C' ，攻击者无法区分哪一个正常承诺（参考 Pedersen 承诺的完美隐藏属性）。□

5.3 威胁模型分析

表 2 威胁模型与防御措施

攻击类型	描述	防御措施
Server 入侵	攻击者获取 server 全部数据	AES 加密 + 无密钥存储 + 承诺仅含 EC 点
份额窃取	攻击者窃取一份或多份份额邮件	信息论安全: $< t$ 份无法恢复
胁迫攻击	攻击者强迫监护人提交份额	胁迫码触发警报并终止继承
中介攻击	攻击者截获份额传输	ECIES 非对称加密传输
刷新攻击	攻击者伪造刷新增量	零和多项式验证 + 承诺一致性检查
重放攻击	攻击者重复使用旧份额	每次刷新后旧承诺失效, Pedersen 验证

5.4 与现有方案的对比

表 3 与现有数字资产继承方案的对比

维度	本方案	Ledger cover	Re- 公证遗嘱	Safe{Wallet}
信任模型	数学（密码学）	中心化服务 + KYC	公证机构	智能合约
中心化程度	去中心化	中心化	中心化	链上多签
隐私保护	全流程加密	KYC 可见	完全暴露	链上公开
资产范围	加密货币 + 云 + 文件	仅 Ledger 设备	通用	仅 EVM
抗胁迫	胁迫码 + 警报	无	无	无
份额刷新	零信任同态刷新	不可刷新	—	不可刷新
成本	无订阅费	\$9.99/月	数千-数万元	Gas 费
开源可审计	全部开源	闭源	—	部分开源

本方案在信任模型（数学去信任 vs 中心化信任）、隐私保护（全流程加密 vs 暴露）、抗胁迫能力和资产覆盖范围方面均优于现有方案。

6 实验评估

6.1 实验环境

表 4 实验环境配置

项目	配置
处理器	12th Gen Intel Core i7-12800HX (24 核)
内存	15.75 GB
操作系统	Windows_NT 10.0.26200
Node.js 版本	v20.19.4 (V8 11.3.244.8)
椭圆曲线库	elliptic.js v6.5.4 (secp256k1)

6.2 功能正确性验证

6.2.1 Shamir 秘密共享正确性

为验证 Shamir 秘密共享实现的正确性,我们在 $n \in \{3, 5, 7, 10\}$ 、 $t \in \{2, 3, 5, 7\}$ 的参数组合下各进行 100 轮随机测试。测试过程为: 随机生成 256 位主密钥 $\rightarrow$ 使用 t -of- n 拆分 $\rightarrow$ 随机选取 t 个份额 $\rightarrow$ Lagrange 插值恢复 $\rightarrow$ 比较与原始主密钥的一致性。

表 5 Shamir 正确性验证结果

n	t	测试轮数	成功次数	正确率	平均耗时 (ms)
3	2	100	100	100.00%	0.0101
5	3	100	100	100.00%	0.0123
7	5	100	100	100.00%	0.0629
10	7	100	100	100.00%	0.0406

6.2.2 同态刷新正确性验证

验证刷新后的份额使用 Lagrange 插值恢复的主密钥与原始主密钥一致。测试参数同上。

验证步骤：（1）生成主密钥 s ；（2）拆分得到 v_i, r_i ；（3）生成零和多项式刷新得到 $\delta_{v,i}, \delta_{r,i}$ ；（4）计算新份额 $v'_i = v_i + \delta_{v,i}$ ；（5）用 t 个 v'_i 恢复 s' ；（6）验证 $s' = s$ 。

表 6 同态刷新正确性验证结果

n	t	测试轮数	刷新前正确率	刷新后正确率	刷新耗时 (ms)
3	2	100	100.00%	100.00%	0.0304
5	3	100	100.00%	100.00%	0.0671
7	5	100	100.00%	100.00%	0.1746
10	7	100	100.00%	100.00%	0.2480

6.3 密码学运算性能测试

表 7 各密码学操作的性能开销

操作	平均耗时 (ms)
主密钥生成（256 位随机数）	0.0206
Pedersen 双多项式拆分（5-of-3）	0.0940
Pedersen 承诺生成	1.3569
零和多项式增量生成（5 份）	0.0693
承诺同态刷新计算（5 份）	6.8089
Lagrange 插值恢复主密钥（3 份）	0.0054
ECIES 加密（份额值）	1.5716
主承诺同态验证（3 份）	1.6624

6.4 规模扩展性测试

测试不同监护人数目 n 和门限 t 下的性能变化趋势（50 轮测试取平均）。

表 8 规模扩展性测试结果

(n, t)	拆分耗时 (ms)	刷新耗时 (ms)	恢复耗时 (ms)
(3, 2)	0.054	0.034	0.004
(5, 3)	0.098	0.125	0.007
(7, 5)	0.175	0.154	0.016
(10, 7)	0.299	0.239	0.032

6.5 同态验证精度测试

测试主承诺同态验证的精度（准确率、召回率）：

表 9 同态验证精度测试

指标	TP	FN	FP	TN	Accuracy
综合（100 轮）	100	0	0	100	100.00%

6.6 与现有方案的对比测试

表 10 与现有方案的对比

特性	本方案	Ledger Recover	Safe{Wallet}
零信任 Server	✓	×	✓
非链上资产支持	✓	×	×
份额刷新	✓（同态）	×	×
抗胁迫	✓	×	×
隐私保护	全流程加密	KYC 可见	链上公开
开源	全栈开源	闭源	部分

实验结论：本文提出的数字遗产管家系统在功能正确性上通过了全部 100 轮测试，Shamir 秘密共享拆分与恢复、同态份额刷新、主承诺同态验证的正确率均达到 100%。密码学操作性能方面，核心操作（承诺生成、同态验证、ECIES 加解密）的平均耗时均在 0.9–6.8ms 范围内，满足实时交互的需求。规模扩展性测试

表明，各操作的耗时随监护人数目呈线性增长趋势，10-of-7 配置下单次拆分仅需 0.3ms，系统具备良好的可扩展性。与现有方案（Ledger Recover、Safe{Wallet}）的对比显示，本方案在零信任 Server、非链上资产支持、份额刷新和抗胁迫设计方面具有显著优势。

7 总结与展望

7.1 工作总结

本文提出并实现了数字遗产管家系统，一个基于门限密码学与同态承诺的可验证数字资产继承协议。主要工作总结如下：

- (1) 设计并实现了双多项式 Shamir 份额结构，在标准 Shamir 方案基础上引入盲因子多项式，使得份额值通过 Pedersen 承诺可在公开存储的同时具备可验证性。
- (2) 提出了基于零和多项式的同态份额刷新协议，在 server 完全不存储主密钥的前提下实现安全的份额轮换，填补了现有方案在零信任密钥管理方面的空白。
- (3) 设计了计算不可区分的胁迫码机制，为数字资产继承领域提供了首创的抗胁迫方案。
- (4) 实现了完整的 Web 系统，覆盖”创建—分发—验证—刷新—恢复”全生命周期，并以 MIT 许可证开源。

7.2 未来工作

本系统在未来可以从以下方向进一步扩展：

分布式密钥生成（DKG）。当前方案在创建阶段需要生成完整主密钥（尽管随后销毁），存在理论上创建者单方面获取密钥的瞬时风险。引入 Feldman DKG 协议 [3]，由 t 位监护人通过交互式协议共同生成主密钥，消除对任何单一实体的信任。

门限签名与链上资产转移。当前资产恢复流程为”解密 → 发送”，对于区块链资产存在二次转移的成本和安全性问题。引入 ECDSA 门限签名协议（如 GG18/GG20 [4]），实现 t 位监护人直接签名链上交易，跳过密钥恢复环节。

零知识证明验证。当前的 Pedersen 承诺同态验证需要公开所有 t 个份额的承诺并进行 Lagrange 系数求和。引入 zk-SNARK/STARK 技术可提供更优的隐私保护和验证效率 [5]。

形式化验证。使用 ProVerif [6] 或 Tamarin [7] 协议验证器对系统的密码学协议进行形式化建模和安全性证明，确保协议的正确性。

抗量子密码学迁移。当前的 secp256k1 曲线可被 Shor 算法在量子计算机上高效破解。随着 NIST 后量子密码标准化进程的推进，将系统迁移到 CRYSTALS-Dilithium [8] 等格基密码方案是长期演进方向。

安全工程强化。包括集成硬件安全模块（HSM）、实现前向安全性、完善侧信道攻击防护和通过第三方安全审计。

参考文献

- [1] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [2] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology—CRYPTO ’91*, pages 129–140. Springer, 1991.
- [3] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science*, pages 427–438. IEEE, 1987.
- [4] Rosario Gennaro and Steven Goldfeder. Threshold ecdsa from ecdsa assumptions: the multiparty case. In *2018 IEEE Symposium on Security and Privacy (S&P)*, pages 1051–1066. IEEE, 2018.
- [5] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, 2018.
- [6] Bruno Blanchet. An efficient cryptographic protocol verifier based on prolog rules. In *14th IEEE Computer Security Foundations Workshop*, pages 82–96. IEEE, 2001.
- [7] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The tamarin prover for the symbolic analysis of security protocols. In *Computer Aided Verification*, pages 696–701. Springer, 2013.
- [8] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022.